

Assignment 1

Deadline: 7 June, 2026 EOD

1 Dataset Preparation & Exploration

Task 1 · RAVDESS Dataset Analysis

The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) contains 2,452 recordings from 24 professional actors (12 male, 12 female) expressing 8 emotions across two modalities (speech and song). Each filename encodes metadata in a structured naming convention.

Dataset: Download from zenodo.org/record/1188976 or via `torchaudio.datasets` (speech-only subset) or via Kaggle.

Part A — Filename Decoding

RAVDESS filenames follow the pattern:

$\underbrace{03}_{\text{modality}} - \underbrace{01}_{\text{vocal channel}} - \underbrace{05}_{\text{emotion}} - \underbrace{01}_{\text{intensity}} - \underbrace{01}_{\text{statement}} - \underbrace{01}_{\text{repetition}} - \underbrace{24}_{\text{actor}} . \text{wav}$

Write a Python function `parse_ravdess_filename(path: str) -> dict` that accepts a file path and returns a dictionary with the keys `modality`, `vocal_channel`, `emotion`, `intensity`, `statement`, `repetition`, and `actor`, mapping each code to its human-readable label.

Part B — Dataset Statistics

1. Load the full speech subset. How many files are there per emotion class? Plot a bar chart of class counts and comment on any class imbalance.
2. What is the mean and standard deviation of audio duration (in seconds) across all files? Plot a histogram of durations.
3. Using `librosa`, load three files—one *happy*, one *sad*, one *angry*—and display their waveforms and mel-spectrograms side by side in a 3×2 subplot grid. Label each plot. What differences, if any, are visually apparent?

Part C — Written Reflection

In 100–150 words, explain why **Unweighted Average Recall (UAR)** is the preferred metric for evaluating emotion recognition on RAVDESS instead of plain accuracy. Include the formula for UAR and explain the consequence of ignoring class imbalance.

2 Acoustic Feature Extraction with openSMILE

Task 2 · eGeMAPS Feature Extraction Pipeline

Install: `pip install opensmile`
openSMILE (*open-Source Media Interpretation by Large feature-space Extraction*) is the industry-standard tool for acoustic feature extraction in affective computing. Its **eGeMAPS** (extended Geneva Minimalistic Acoustic Parameter Set) extracts 88 functionals designed specifically for speech and emotion research.

Part A — Single-File Extraction

```
1 import opensmile
2
3 smile = opensmile.Smile(
4     feature_set=opensmile.FeatureSet.eGeMAPSv02,
5     feature_level=opensmile.FeatureLevel.Functionals,
6 )
7 features = smile.process_file("path/to/file.wav")
8 print(features.shape) # should be (1, 88)
9 print(features.columns.tolist())
```

1. Run the above on a single RAVDESS .wav file. Print the 88 feature names and their values.
2. Choose any **three** eGeMAPS features (e.g. `F0semitoneFrom27.5Hz_sma3nz_amean`, `loudness_sma3_amean`, `mfcc1_sma3_amean`). In one sentence each, describe what each feature measures acoustically.

Part B — Full Dataset Extraction

Write a script that iterates over all RAVDESS speech files, extracts eGeMAPS functionals for each, attaches the emotion label parsed from the filename, and saves the result to `ravdess_egemaps.csv`. Include a progress bar (`tqdm` recommended). The resulting CSV should have 88 feature columns plus a `label` column. Report the final shape of the DataFrame.

Part C — Feature-Level Analysis

1. Compute the per-class mean of `F0semitoneFrom27.5Hz_sma3nz_amean` (mean pitch) across all emotion classes and plot as a horizontal bar chart. Which emotion has the highest mean pitch? Is this consistent with your intuition?
2. Compute the correlation matrix of the 88 features and display it as a heatmap. Identify and name at least two pairs of highly correlated features ($|r| > 0.9$).

3 Classical ML Baselines

Task 3 · SVM & Random Forest Classifiers

Using the `ravdess_egemaps.csv` produced in Task 2, train and evaluate two classical baseline classifiers with proper cross-validation.

Part A — SVM Baseline

1. Standardise your features using `StandardScaler`. **Why is this important for SVMs?** (2–3 sentences.)
2. Train a `SVC` with an RBF kernel using **10-fold stratified cross-validation**. Report the mean and standard deviation of both accuracy and UAR across folds.
3. Plot the confusion matrix (averaged or from the final fold). Which emotion pairs are most often confused? Offer a brief acoustic explanation.
4. Try at least two different values for the regularisation parameter C (e.g. $C \in \{0.1, 1, 10\}$) and compare UAR. No need for exhaustive grid search—just show the effect of C qualitatively.

Part B — Random Forest Baseline

1. Train a `RandomForestClassifier` with `n_estimators=200` using the same 10-fold CV setup. Report mean UAR and compare with the SVM.
2. Extract feature importances from the trained forest. Plot the **top-20 most important eGeMAPS features** as a horizontal bar chart, coloured by feature group (e.g. pitch, energy, spectral, MFCCs).
3. Are the important features consistent with what you know about acoustic correlates of emotion? Discuss in 80–120 words.

Part C — Model Comparison Table

Fill in a table of the following form in your notebook:

Model	Features	Mean Acc.	Mean UAR	UAR Std
SVM (RBF)	eGeMAPS			
Random Forest	eGeMAPS			

Which model do you recommend as a baseline for the AudialMind project? Justify your choice in 2–3 sentences.

4 Feature Space Visualisation with UMAP

Task 4 · UMAP Dimensionality Reduction

Install: `pip install umap-learn`

UMAP (Uniform Manifold Approximation and Projection) projects high-dimensional feature vectors into 2D while preserving both local and global structure better than t-SNE for large datasets.

Part A — 2D Projection

```
1 import umap
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv("ravdess_egemaps.csv")
6 X = df.drop("label", axis=1).values
7 y = df["label"].values
8
9 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state
10                     =42)
11 embedding = reducer.fit_transform(X)
12
13 # Scatter plot coloured by emotion label
```

1. Produce a scatter plot of the 2D embedding, coloured by emotion class. Use a distinct colour per class and include a legend.
2. Do emotion clusters appear in the UMAP plot? Describe the structure you observe (e.g. which emotions cluster together, which overlap).
3. Experiment with `n_neighbors` $\in \{5, 15, 30\}$. How does changing this hyperparameter affect the visual clustering? Include all three plots.

Part B — Standardised vs Raw Features

1. Run UMAP on both raw and `StandardScaler`-normalised features. Plot both side by side and comment on any differences.
2. In 60–100 words, explain why dimensionality reduction is useful for understanding your feature set before training a classifier.

5 CNN Spectrogram Classifier

Task 5 · Minimal CNN on Mel-Spectrograms

Building on the idea that a mel-spectrogram is just an image, you will train a small convolutional neural network to classify RAVDESS emotions directly from spectrograms.

Part A — Data Pipeline

1. Using `torchaudio`, write a `Dataset` class that:
 - loads a `.wav` file,
 - resamples to 22,050 Hz if needed,
 - computes a **mel-spectrogram** with `n_mels=64`,
 - applies `AmplitudeToDB`, and
 - pads or crops to a fixed time dimension of 128 frames.
2. Split the dataset into 80% train / 20% test (stratified by emotion). Report the number of samples in each split.

Part B — Model Architecture

Define a CNN in PyTorch with at least:

- two convolutional blocks (`Conv2d` → `BatchNorm` → `ReLU` → `MaxPool`),
- a global average pooling layer, and
- a linear classifier head.

Print the model summary (parameter count). You may use `torchinfo` or a manual count. Your model should have **fewer than 500,000 parameters** to keep training tractable.

Part C — Training & Evaluation

1. Train for 20 epochs using the `Adam` optimiser (`lr=1e-3`) and `CrossEntropyLoss`. Plot the training loss and validation loss curves.
2. Report the final test UAR and accuracy. How does the CNN compare to the classical baselines from Task 3?
3. In 80–100 words, discuss one advantage and one disadvantage of the spectrogram-as-image approach compared to hand-crafted eGeMAPS features.

Bonus Task · Write Your Own openSMILE Config File

openSMILE is built around a modular **component-and-dataflow architecture** specified entirely in `.conf` configuration files. Each config wires together *sources* (e.g. a waveform reader), *processing components* (e.g. windowing, FFT, filterbanks), and *sinks* (e.g. a CSV writer). This bonus task asks you to understand that architecture and create a small custom config from scratch.

Step 1 — Required Reading (ungraded)

Before writing any code, work through the following:

1. **openSMILE Documentation** — Read the [audeering Python wrapper docs](#) and skim the [README](#) on GitHub.
2. **Configuration Architecture** — Read the section on the *component manager* and *dataMemory* in the original openSMILE book (freely available at audeering.com/opensmile). Pay particular attention to how `[componentInstances:cComponentManager]` wires components together.
3. **Built-in Config Reference** — Study the existing `egemaps02.conf` file (find it inside your installed `opensmile` package via `import opensmile; print(opensmile._data_dir)`). Annotate it: what does each component do?

Step 2 — Write a Custom Config

Create a file named `my_features.conf` that extracts a **custom feature set of your own design**. Your config must:

1. Wire together at least **four distinct components**, which must include:
 - a waveform source (`cWaveSource`),
 - a framing / windowing component (`cFramer`, `cWindower`),
 - at least one spectral or cepstral feature extractor (e.g. `cMfcc`, `cPitch`, `cSpectral`), and
 - a CSV or ARFF sink (`cCsvSink` or `cArffSink`).
2. Differ meaningfully from the built-in `eGeMAPSv02` config—for example by using a different frame size, different number of MFCC coefficients, or adding a feature group that `eGeMAPS` omits (e.g. chroma features, spectral flux, zero-crossing rate).

Step 3 — Evaluation

1. Load your custom config via:

```
1 smile_custom = opensmile.Smile(
2     feature_set="my_features.conf",
3     feature_level=opensmile.FeatureLevel.Functionals,
4 )
5 df_custom = smile_custom.process_file("sample.wav")
6 print(df_custom.shape, df_custom.columns.tolist())
```

2. Extract your custom features for the full RAVDESS speech set and train a quick SVM baseline (same 10-fold CV). Report the UAR.

Deliverable: Submit `my_features.conf` alongside your notebook.

Submission Guidelines

- Create your Github repository for the AudialMind. This would be the primary destination of your assignment submissions and even the final project.
- Submit a single **Jupyter Notebook** (`assignment1_YourName.ipynb`) with all cells executed and outputs visible in a folder name 'Assignment1' on your repo.
- Keep all file paths **relative** (e.g. `./data/ravdess/`). Do not hardcode absolute paths.
- If you complete the bonus task, include `my_features.conf` and Jupyter Notebook containing the trained model and test results in `/Assignment1/bonus/` folder in your submission zip.
- Plots must have **axis labels**, **titles**, and **legends** where applicable.
- Create a report comprising of all the results and the answer to the questions mentioned in each task. It should contain all the plots also **It should not be AI generated**.